

Invisible Cloak Using Python & OpenCV

Complete Project Process, Tools Used, System Architecture and DFD

Project Type: Computer Vision / Image Processing

Main Technology: Python + OpenCV + NumPy

Input: Live webcam video and a colored cloak/cloth

Output: Real-time video in which the selected cloak area is replaced by the previously captured background

1. Project Objective

The objective is to create an 'invisible cloak' effect using a webcam. The program first captures the empty background, then detects the cloak color in each live frame. Wherever the cloak is detected, that region is replaced with the corresponding region from the stored background image.

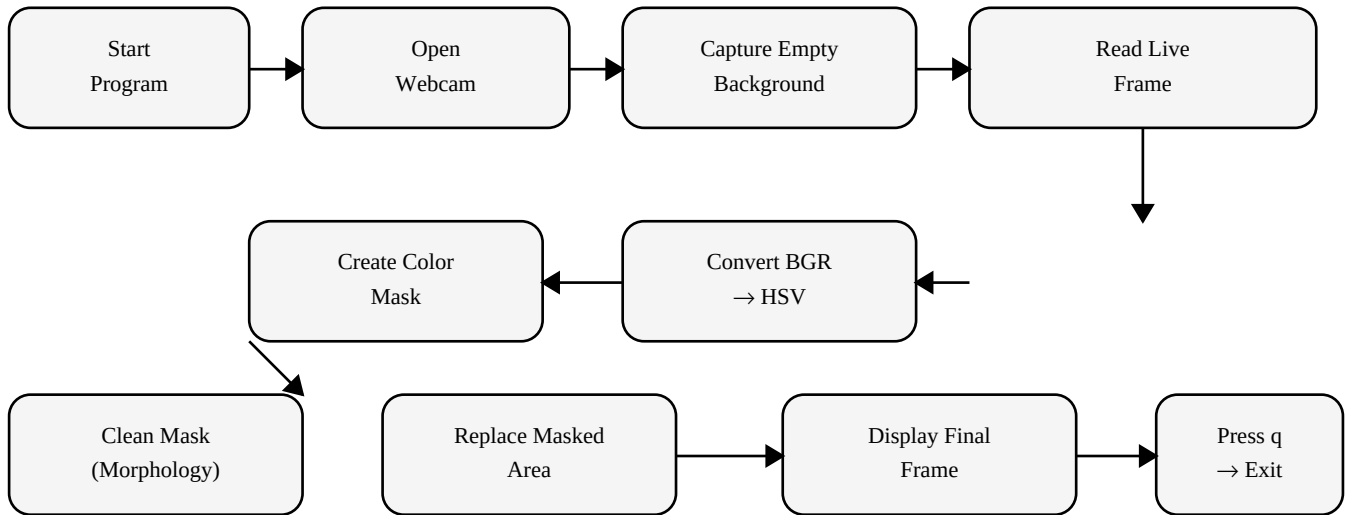
This is an educational computer-vision project. It does not make the cloth physically invisible; it creates the visual illusion by replacing pixels.

2. Tools and Technologies Used

Tool / Technology	Purpose
Python 3	Programming language used to build the application.
Visual Studio Code	Code editor used to write, save and run the Python program.
OpenCV (cv2)	Webcam capture, color conversion, masking, morphology and image processing.
NumPy	Creates and manipulates image arrays and mask matrices.
Webcam	Provides the live video frames.
HSV Color Space	Makes color-based segmentation easier than direct BGR/RGB thresholding.

3. Overall Working Process

The project follows these major stages:



Important: the empty background must remain mostly unchanged during background capture. The effect works best with a fixed camera and a cloak whose color is clearly different from the surrounding background.

4. Step-by-Step Development Process

Step 1 — Install Python

Install Python 3 on Windows. During installation, enable **Add Python to PATH**. Verify with `python --version` in the VS Code terminal.

Step 2 — Install VS Code and the Python Extension

Install Visual Studio Code, open the project folder, and install the Microsoft Python extension from the Extensions panel.

Step 3 — Create the Project

Create a folder named **InvisibleCloak** and create the file `invisible_cloak.py` inside it.

Step 4 — Install Python Libraries

Run this command in the VS Code terminal:

```
pip install opencv-python numpy
```

Step 5 — Start the Webcam

OpenCV uses `cv2.VideoCapture(0)` to connect to the default webcam. The program reads frames continuously.

Step 6 — Capture the Background

Before the person enters the scene, several webcam frames are read and one frame is stored as the background. This image is later used to replace the cloak region.

Step 7 — Convert BGR to HSV

OpenCV normally reads webcam frames as BGR. The program converts each frame to HSV using `cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`. HSV is convenient for selecting a color range.

Step 8 — Create the Color Mask

The program defines lower and upper HSV limits for the cloak color and uses `cv2.inRange()`. Pixels inside the selected color range become white in the mask; other pixels become black.

For a red cloak, two ranges are commonly used because red appears at both ends of the HSV hue scale.

Step 9 — Clean the Mask

Morphological opening removes small noise, while dilation can make the detected cloak region more continuous.

Step 10 — Create the Inverse Mask

The inverse mask identifies the part of the current frame that is not the cloak. This part is retained normally.

Step 11 — Replace the Cloak with Background

The current-frame non-cloak region and the stored-background cloak region are combined using bitwise operations. The result creates the invisible effect.

Step 12 — Display the Result

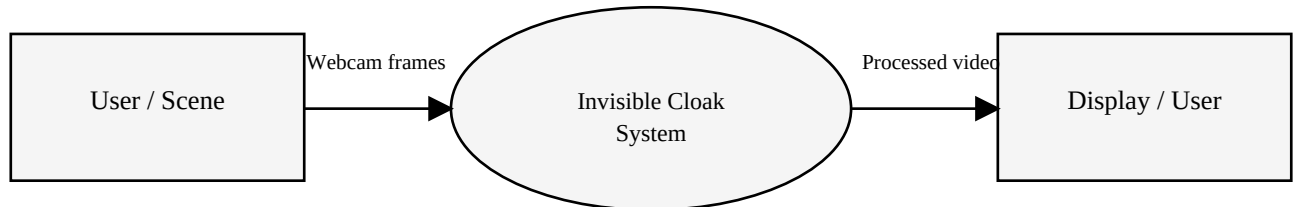
The final frame is shown using **cv2.imshow()**. Press **q** to stop the program, then release the webcam and close OpenCV windows.

5. Core OpenCV Functions Used

Function	Use
cv2.VideoCapture(0)	Connect to webcam.
cap.read()	Read a video frame.
cv2.flip()	Mirror the frame for a natural webcam view.
cv2.cvtColor()	Convert BGR image to HSV.
cv2.inRange()	Create a binary color mask.
cv2.morphologyEx()	Remove small noise / improve mask.
cv2.bitwise_and()	Extract selected regions using masks.
cv2.bitwise_not()	Invert a mask.
cv2.addWeighted()	Combine image regions.
cv2.imshow()	Display the processed video.

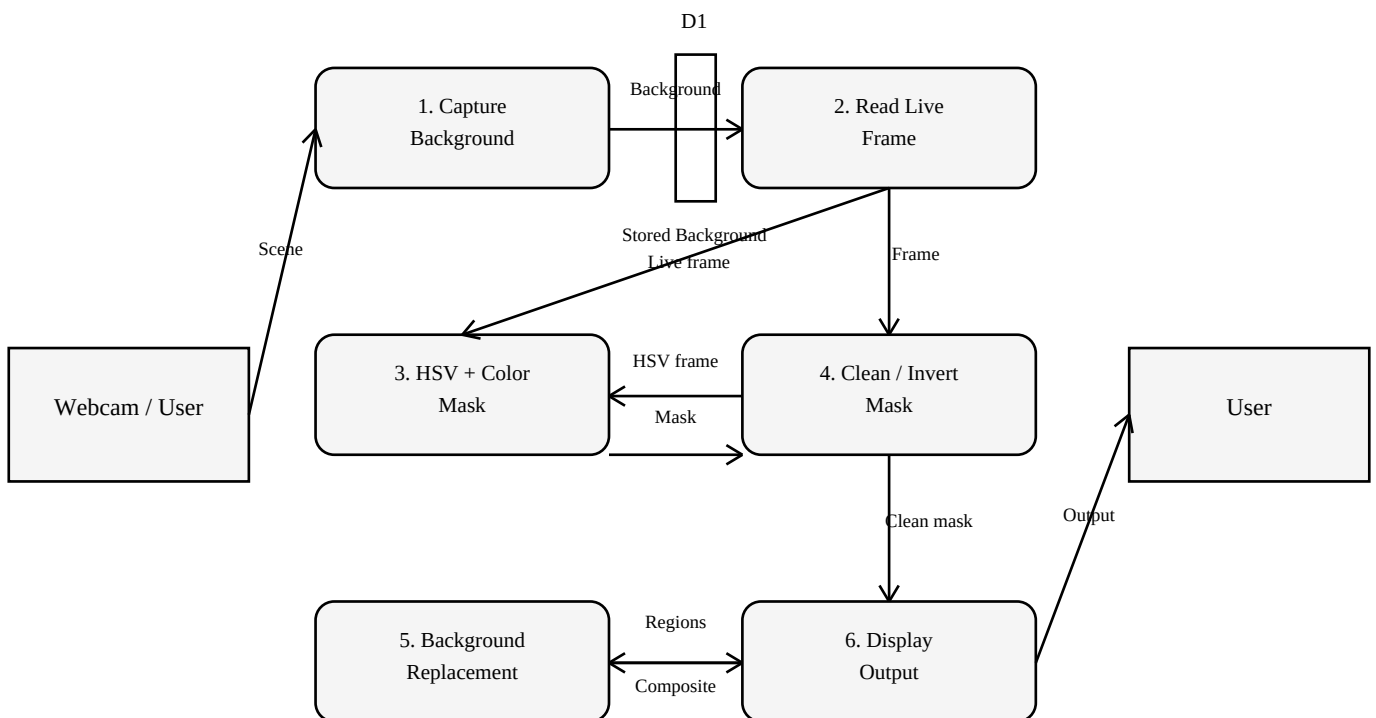
6. Data Flow Diagram (DFD) — Level 0 / Context Diagram

The Level-0 DFD treats the Invisible Cloak application as one main process. The user provides the webcam scene and cloak, while the system produces the processed video output.



7. DFD — Level 1

Level 1 expands the main system into its major processing steps: background capture, frame acquisition, color detection, mask processing, background replacement, and output display.



DFD Data Stores and Flows

Symbol / ID	Meaning
External Entity	Webcam / user provides the scene and cloak.
P1	Capture and store the empty background.
P2	Read each live webcam frame.
P3	Convert frame to HSV and detect cloak color.
P4	Clean and invert the binary mask.
P5	Use the mask to replace cloak pixels with background pixels.
P6	Display the final processed frame.
D1	Stored background image used for replacement.

8. Algorithm / Pseudocode

1. Start webcam.
2. Capture an empty background frame.
3. Read a new webcam frame continuously.
4. Flip the frame horizontally.
5. Convert BGR to HSV.
6. Define the HSV range for the cloak color.
7. Create the color mask with `inRange()`.
8. Remove noise using morphology.
9. Invert the mask.
10. Extract the normal region from the current frame.
11. Extract the corresponding region from the stored background.
12. Combine both regions.
13. Display the final result.
14. Stop when the user presses `q`.

9. Example Project Folder

```
InvisibleCloak/  
■■■ invisible_cloak.py
```

10. How to Run

Open the InvisibleCloak folder in VS Code. Open Terminal → New Terminal and run:

```
python invisible_cloak.py
```

Keep the camera view empty during the initial background capture. Then enter the scene wearing the selected colored cloak. Press `q` to exit.

11. Common Problems and Solutions

Problem	Solution
Camera does not open	Check camera permission, close other camera applications, and try <code>cv2.VideoCapture(0)</code> or another
<code>ModuleNotFoundError: cv2</code>	Run: <code>pip install opencv-python</code>
<code>ModuleNotFoundError: numpy</code>	Run: <code>pip install numpy</code>
Cloak is not detected	Use a brighter, more distinct cloak color and adjust HSV lower/upper values.
Background appears wrong	Keep the camera fixed and do not stand in the scene while the background is captured.
Mask has noise	Adjust HSV thresholds and use morphological opening/dilation.

12. Limitations

The effect depends on stable lighting, a mostly static camera, a clear cloak color, and a background that remains sufficiently similar to the captured background. Shadows, reflections, skin/clothing with similar colors, and sudden camera movement can reduce the quality of the effect.

13. Possible Future Improvements

- Add HSV trackbars for interactive color selection.
- Support multiple cloak colors.
- Add a GUI for camera and color settings.

- Improve segmentation using adaptive thresholds or machine-learning-based segmentation.
- Save screenshots or recorded output video.
- Add a start/stop button and status indicators.

14. Conclusion

The Invisible Cloak project demonstrates an end-to-end computer-vision pipeline using Python, OpenCV and NumPy. The key concept is color-based segmentation: detect the cloak, create a mask, and replace that masked area with the previously captured background. It is a useful practical project for learning image processing, masks, HSV color space, webcam handling and real-time OpenCV operations.